Chennai BnB Property Management System

Software Documentation & Automated Test Report (Assignment Parts A & B)

Name	**Jeevan S**
Registration No	**24BCE1862**
Course Code	**BCSE301L**
Semester	**FALL26-27**
Assessment	**SWE DA3**

Generated 2026-10-21 | 30 automated test cases | execution time 20.14s

30	24	6	6	3
TEST CASES	PASSED	FAILED	DEFECTS FOUND	DEFECTS FIXED

Contents

1. The Software
2. Architecture & Data Model
3. Test Strategy & Tooling
4. Overall Test Results
5. Test Cases & Evidence (by category)
6. Defects Found
7. Defects Found & Fixed
8. How to Run the Demo

1. The Software

Chennai BnB PMS is a Property Management System for a small 8-room serviced-apartment property. It lets front-desk staff manage rooms, guests and the complete booking lifecycle, and exposes the same data through a REST API.

TECHNOLOGY STACK

Layer	Technology
Backend	Python 3 / Flask (server-rendered Jinja2 templates + REST API)

Database	MySQL 8.0 via `mysql-connector-python`
Frontend	HTML5 + CSS (Jinja2 templates), vanilla JavaScript
Auth	Flask server-side sessions, role-based access control
Test automation	pytest + requests (API/DB), Playwright (browser UI)

CORE FEATURES

- **Authentication & roles** - three roles (manager, reception, auditor) with per-endpoint authorisation.
- **Dashboard** - live occupancy rate, available/occupied counts, today's check-ins and check-outs.
- **Room management** - list all rooms, edit price and status (available / occupied / maintenance).
- **Guest management** - guest records created automatically from bookings, with contact details and ID proof.
- **Booking lifecycle** - create a booking, check in, check out or cancel, with automatic room-status cascade.
- **Availability engine** - date-range overlap detection prevents double-booking; maintenance rooms are excluded.
- **REST API** - 11 JSON endpoints covering rooms, bookings, guests, availability and occupancy reporting.
- **Security hardening** - CSP, X-Frame-Options, nosniff, Referrer-Policy and no-store headers on every response.

USER ROLES

Role	Login	Permissions
Manager	`manager / manager123`	Full access including booking mutation
Reception	`reception / reception123`	Bookings, guests, rooms, availability
Auditor	`auditor / auditor123`	Read-only reporting; blocked from mutating bookings

2. Architecture & Data Model

The application is a single Flask service. Browser requests are served as Jinja2-rendered HTML; the same `Database` layer also backs the JSON API, so the UI and the API always read and write identical records in MySQL.

> **Request flow:** Browser / API client → Flask route (auth decorator) → validation → `Database` class → MySQL → JSON or rendered template.

DATABASE SCHEMA

Table	Key columns	Notes

`rooms`	room_id, room_number (UNIQUE), room_type, base_price, status, floor, current_guest_id, current_booking_id	8 seeded rooms; status is an ENUM
`guests`	guest_id, name, email, phone, id_proof, created_at	Created automatically when a booking is made
`bookings`	booking_id, room_id (FK), guest_id (FK), check_in, check_out, total_price, status, checked_in_at, checked_out_at	FKs cascade on delete; status is an ENUM

BOOKING LIFECYCLE

`confirmed` → `checked_in` → `checked_out`, with `cancelled` reachable from any active state. Creating a booking marks the room `occupied`; checking out or cancelling returns it to `available` - but only when no other active booking still holds that room.

REST API ENDPOINTS

Method	Endpoint	Auth
GET	`/api/hotel-info`	none
GET	`/api/rooms`	none
GET, PUT	`/api/rooms/<id>`	none
GET	`/api/rooms/<id>/guest`	any role
GET	`/api/rooms/available`	any role
GET, POST	`/api/bookings`	none
GET, PUT, DELETE	`/api/bookings/<id>`	manager, reception
GET	`/api/guests`	any role
GET	`/api/guests/<id>`	any role
GET	`/api/guests/checked-in`	any role
GET	`/api/occupancy`	any role

3. Test Strategy & Tooling

30 automated test cases were written and are organised into the three categories required by the assignment. Tests run at two layers so the same system is verified from the outside (browser) and at the contract level (HTTP + SQL).

Layer	Tooling	What it covers
UI	Playwright (Chromium)	Real browser journeys; every test captures a screenshot
API	pytest + requests	Status codes, JSON payloads, validation and RBAC
Database	mysql-connector	Direct SQL assertions that data really persisted

TEST ISOLATION

Every test runs against a database reset to the identical 8-room seed state, so results are deterministic and the suite can be re-run live, repeatedly, without drift. The Flask server is started automatically by the test session.

4. Overall Test Results

Category	Total	Passed	Failed	Failing cases
Functional End-to-End (E2E) Paths	8	7	1	E2E-08
Boundary & Edge Cases	13	10	3	BND-05, BND-11, BND-12
Integration Points	9	7	2	INT-08, INT-09
Total	**30**	**24**	**6**	

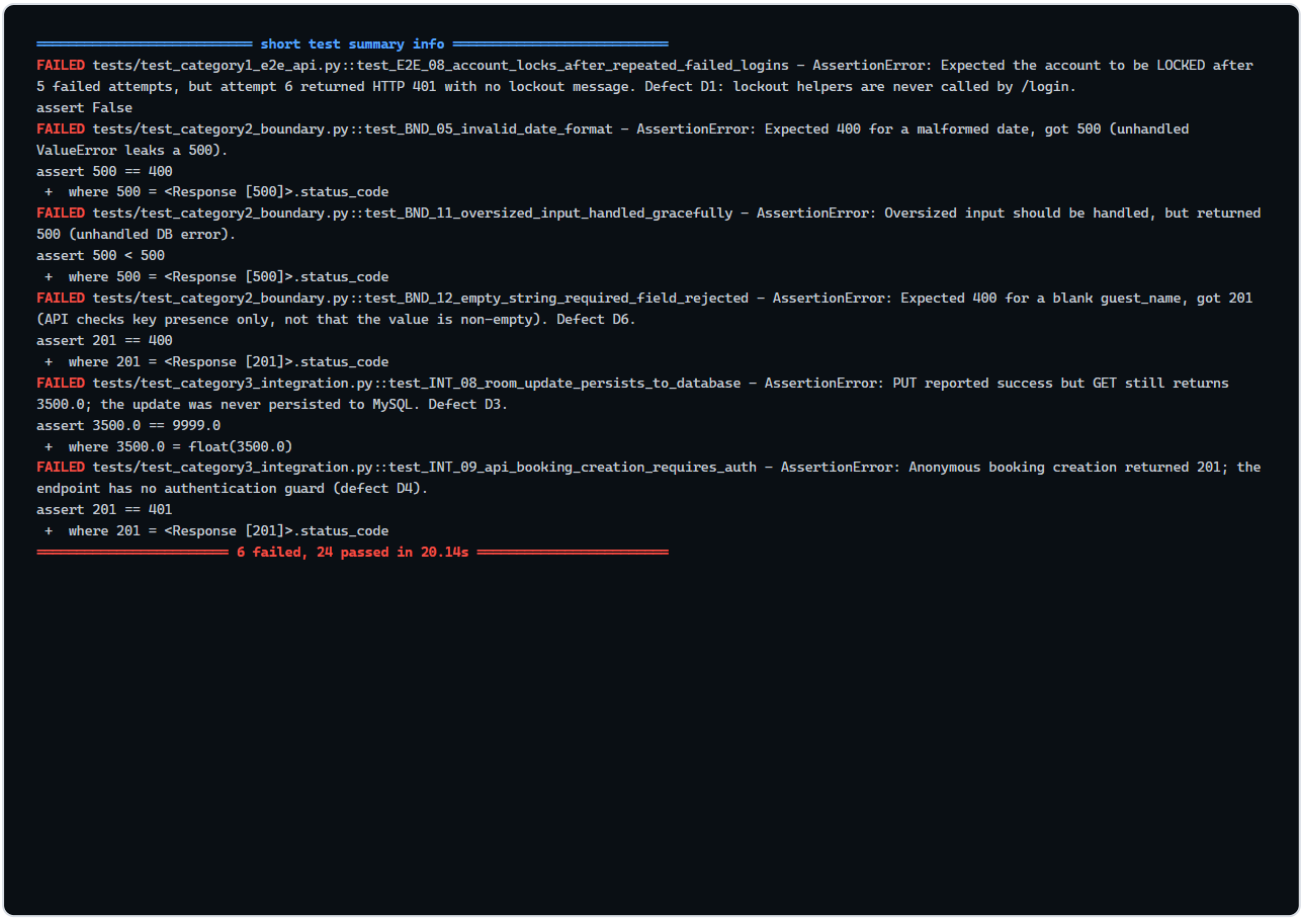
> **Reading these results:** the 6 failures are intentional. They are genuine defects left in the build so the automation can be seen catching real bugs, with at least one failure in every category. A further 3 defects were found during this exercise and fixed - the tests covering them now pass, which is what confirms the fixes.

```
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Python314\python.exe
metadata: {'Python': '3.14.3', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'anyio':
'4.13.0', 'locust': '2.46.7', 'html': '4.2.0', 'metadata': '3.1.1', 'playwright': '0.10.0'}}
rootdir: D:\everything\cyberattacks\dummy-pms
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.13.0, locust-2.46.7, html-4.2.0, metadata-3.1.1, playwright-0.10.0
collecting ... collected 30 items

tests/test_category1_e2e_api.py::test_E2E_02_protected_page_requires_login PASSED [  3%]
tests/test_category1_e2e_api.py::test_E2E_06_api_create_booking PASSED   [  6%]
tests/test_category1_e2e_api.py::test_E2E_07_checked_in_guests_report PASSED [ 10%]
tests/test_category1_e2e_api.py::test_E2E_08_account_locks_after_repeated_failed_logins FAILED [ 13%]
tests/test_category1_e2e_ui.py::test_E2E_01_login_as_manager[chromium] PASSED [ 16%]
tests/test_category1_e2e_ui.py::test_E2E_03_create_booking_through_the_form[chromium] PASSED [ 20%]
tests/test_category1_e2e_ui.py::test_E2E_04_full_stay_lifecycle[chromium] PASSED [ 23%]
tests/test_category1_e2e_ui.py::test_E2E_05_edit_room_price_persists[chromium] PASSED [ 26%]
tests/test_category1_e2e_ui.py::test_INT_05_ui_write_is_visible_through_the_api[chromium] PASSED [ 30%]
tests/test_category2_boundary.py::test_BND_01_login_wrong_password PASSED [ 33%]
tests/test_category2_boundary.py::test_BND_02_open_redirect_guard_on_next_param PASSED [ 36%]
tests/test_category2_boundary.py::test_BND_03_checkout_equals_checkin PASSED [ 40%]
tests/test_category2_boundary.py::test_BND_04_checkout_before_checkin PASSED [ 43%]
tests/test_category2_boundary.py::test_BND_05_invalid_date_format FAILED [ 46%]
tests/test_category2_boundary.py::test_BND_06_missing_required_field PASSED [ 50%]
tests/test_category2_boundary.py::test_BND_07_nonexistent_room PASSED    [ 53%]
tests/test_category2_boundary.py::test_BND_08_double_booking_overlap_rejected PASSED [ 56%]
tests/test_category2_boundary.py::test_BND_09_room_price_zero_rejected PASSED [ 60%]
tests/test_category2_boundary.py::test_BND_10_room_number_uniqueness_enforced PASSED [ 63%]
tests/test_category2_boundary.py::test_BND_11_oversized_input_handled_gracefully FAILED [ 66%]
tests/test_category2_boundary.py::test_BND_12_empty_string_required_field_rejected FAILED [ 70%]
tests/test_category2_boundary.py::test_BND_13_booking_form_rejects_reversed_dates PASSED [ 73%]
tests/test_category3_integration.py::test_INT_01_booking_persists_to_database PASSED [ 76%]
tests/test_category3_integration.py::test_INT_02_booking_sets_room_occupied PASSED [ 80%]
tests/test_category3_integration.py::test_INT_03_checkout_frees_the_room PASSED [ 83%]
tests/test_category3_integration.py::test_INT_04_total_price_computed_from_room_and_nights PASSED [ 86%]
tests/test_category3_integration.py::test_INT_06_auditor_cannot_mutate_bookings PASSED [ 90%]
tests/test_category3_integration.py::test_INT_07_unauthenticated_api_access_blocked PASSED [ 93%]
tests/test_category3_integration.py::test_INT_08_room_update_persists_to_database FAILED [ 96%]
tests/test_category3_integration.py::test_INT_09_api_booking_creation_requires_auth FAILED [100%]

================================== FAILURES ===================================
E   AssertionError: Expected the account to be LOCKED after 5 failed attempts, but attempt 6 returned HTTP 401 with no lockout message. Defect D1:
lockout helpers are never called by /login.
    assert False
D:\everything\cyberattacks\dummy-pms\tests\test_category1_e2e_api.py:67: AssertionError: Expected the account to be LOCKED after 5 failed attempts, but
attempt 6 returned HTTP 401 with no lockout message. Defect D1: lockout helpers are never called by /login.
E   AssertionError: Expected 400 for a malformed date, got 500 (unhandled ValueError leaks a 500).
    assert 500 == 400
     +  where 500 = <Response [500]>.status_code
D:\everything\cyberattacks\dummy-pms\tests\test_category2_boundary.py:61: AssertionError: Expected 400 for a malformed date, got 500 (unhandled
ValueError leaks a 500).
E   AssertionError: Oversized input should be handled, but returned 500 (unhandled DB error).
    assert 500 < 500
     +  where 500 = <Response [500]>.status_code
D:\everything\cyberattacks\dummy-pms\tests\test_category2_boundary.py:133: AssertionError: Oversized input should be handled, but returned 500 (unhandled
DB error).
E   AssertionError: Expected 400 for a blank guest_name, got 201 (API checks key presence only, not that the value is non-empty). Defect D6.
    assert 201 == 400
     +  where 201 = <Response [201]>.status_code
D:\everything\cyberattacks\dummy-pms\tests\test_category2_boundary.py:149: AssertionError: Expected 400 for a blank guest_name, got 201 (API checks key
presence only, not that the value is non-empty). Defect D6.
E   AssertionError: PUT reported success but GET still returns 3500.0; the update was never persisted to MySQL. Defect D3.
    assert 3500.0 == 9999.0
     +  where 3500.0 = float(3500.0)
D:\everything\cyberattacks\dummy-pms\tests\test_category3_integration.py:104: AssertionError: PUT reported success but GET still returns 3500.0; the
update was never persisted to MySQL. Defect D3.
E   AssertionError: Anonymous booking creation returned 201; the endpoint has no authentication guard (defect D4).
    assert 201 == 401
     +  where 201 = <Response [201]>.status_code
D:\everything\cyberattacks\dummy-pms\tests\test_category3_integration.py:119: AssertionError: Anonymous booking creation returned 201; the endpoint has
no authentication guard (defect D4).
- Generated html report: file:///D:/everything/cyberattacks/dummy-pms/test_evidence/report.html -
```

Full pytest run - every test case with its result.



Failure summary: each failing assertion names the defect.

5. Test Cases & Evidence

Every test case is listed below with its purpose, automation layer and the actual result from the run above. **All 30 test cases carry their own visual evidence**, captured automatically while the suite was executing:

- **UI test cases** - full-page screenshots of the real Chromium browser at each step of the journey.
- **API and database test cases** - an evidence card showing the actual HTTP request and response recorded during the run (method, URL, payload, status code and JSON body), the real assertion lines taken from the test source, and the outcome. Nothing on these cards is written by hand; it is the live exchange from the run above.

Category 1 — Functional End-to-End (E2E) Paths

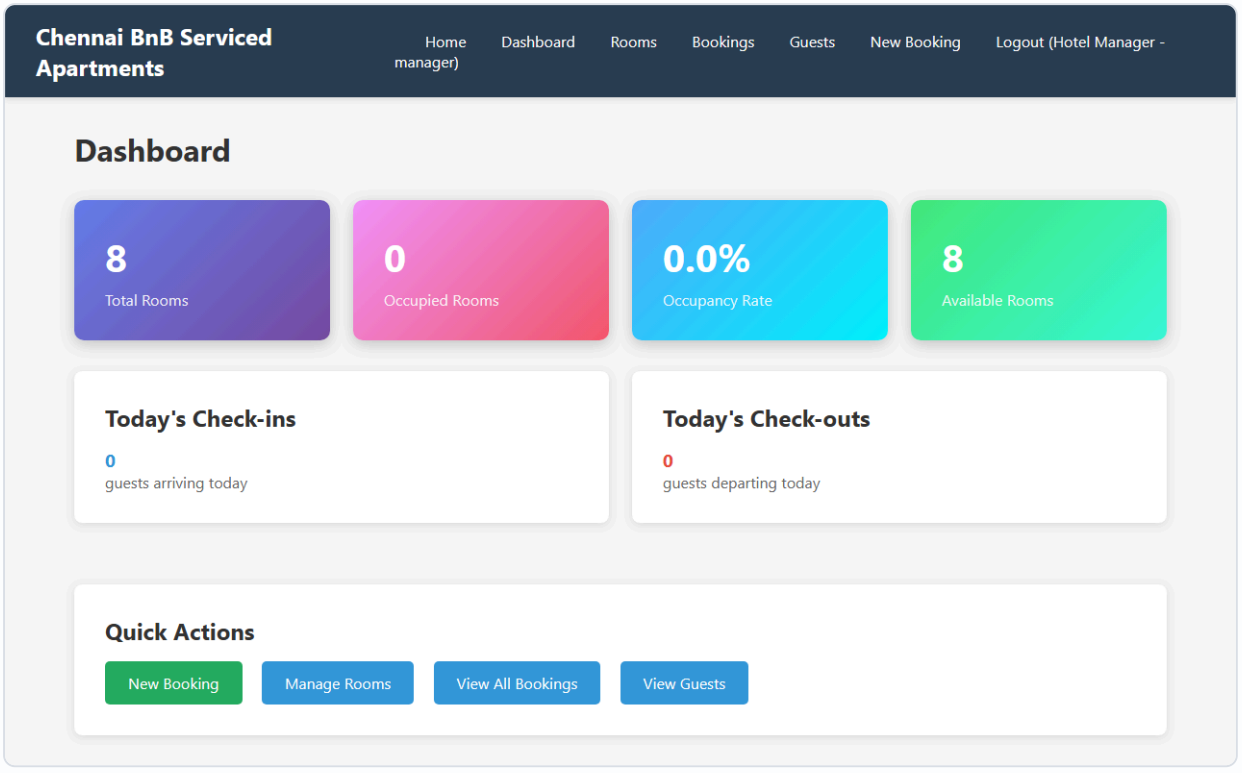
Critical user journeys verified through the browser UI and the REST API.

ID	Test case	Layer	Result
`E2E-01`	Login as manager	UI	PASSED
`E2E-02`	Protected page requires login	API	PASSED

`E2E-03`	Create booking through the form	UI	PASSED
`E2E-04`	Full stay lifecycle	UI	PASSED
`E2E-05`	Edit room price persists	UI	PASSED
`E2E-06`	API create booking	API	PASSED
`E2E-07`	Checked-in guests report	API	PASSED
`E2E-08`	Brute-force lockout journey	API	FAILED

E2E-01 — Login as manager PASSED

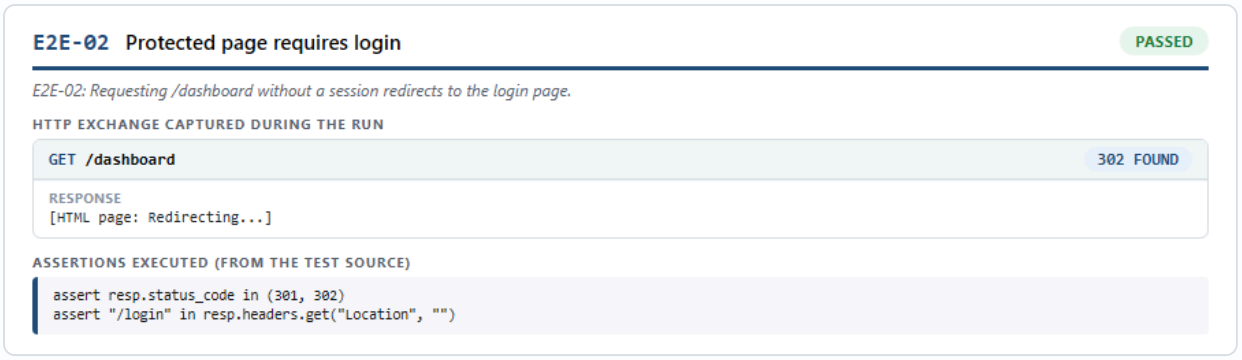
Layer: UI | Valid credentials authenticate and land on the dashboard.



E2E-01 — Dashboard after login

E2E-02 — Protected page requires login PASSED

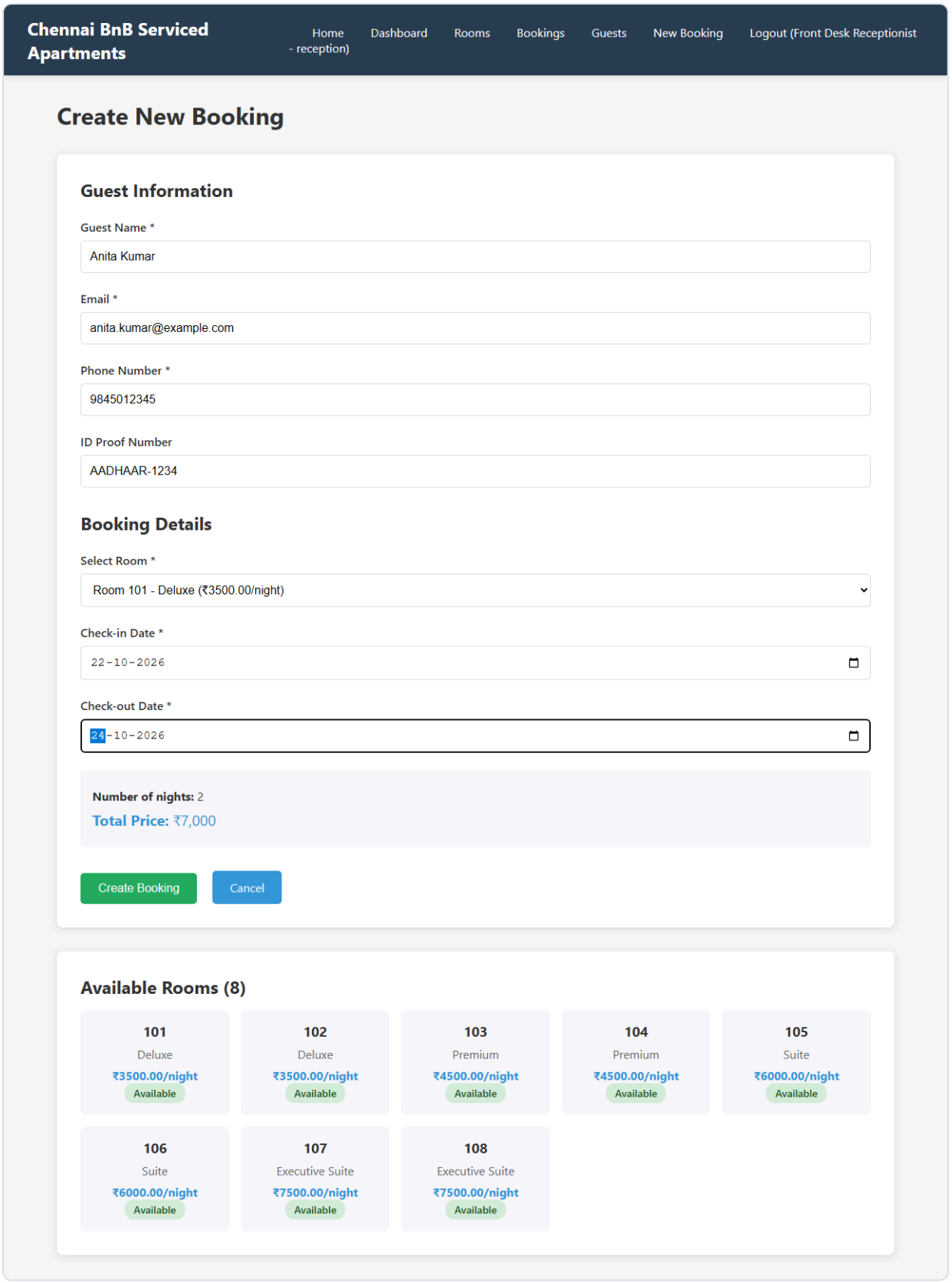
Layer: API | Anonymous /dashboard request redirects to /login.



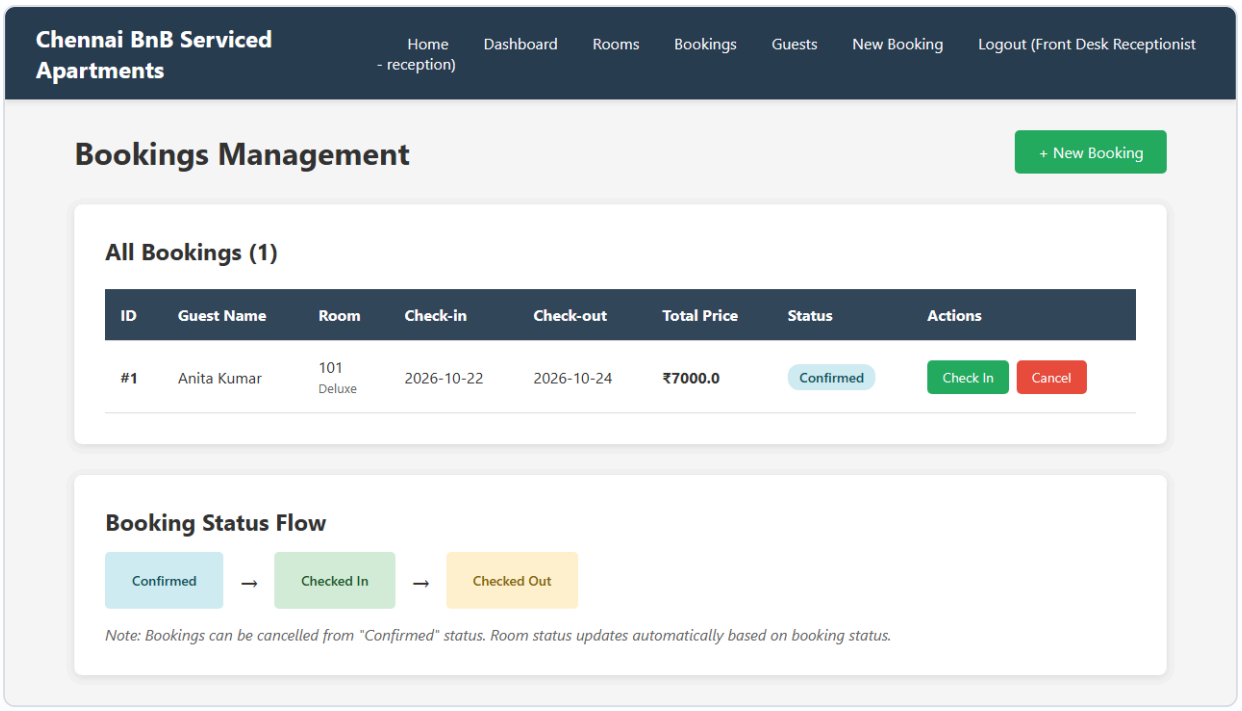
E2E-02 — Api evidence

E2E-03 — Create booking through the form PASSED

Layer: UI | Full booking creation journey through the UI form.



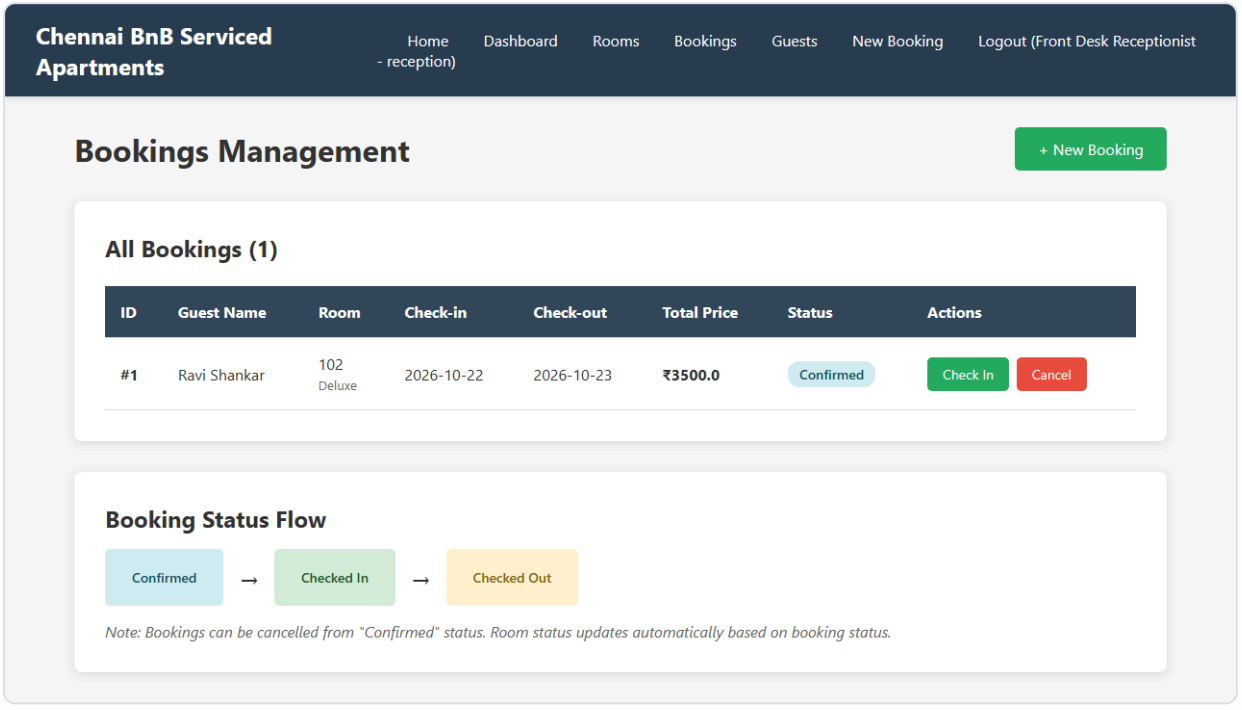
E2E-03 — Booking form filled



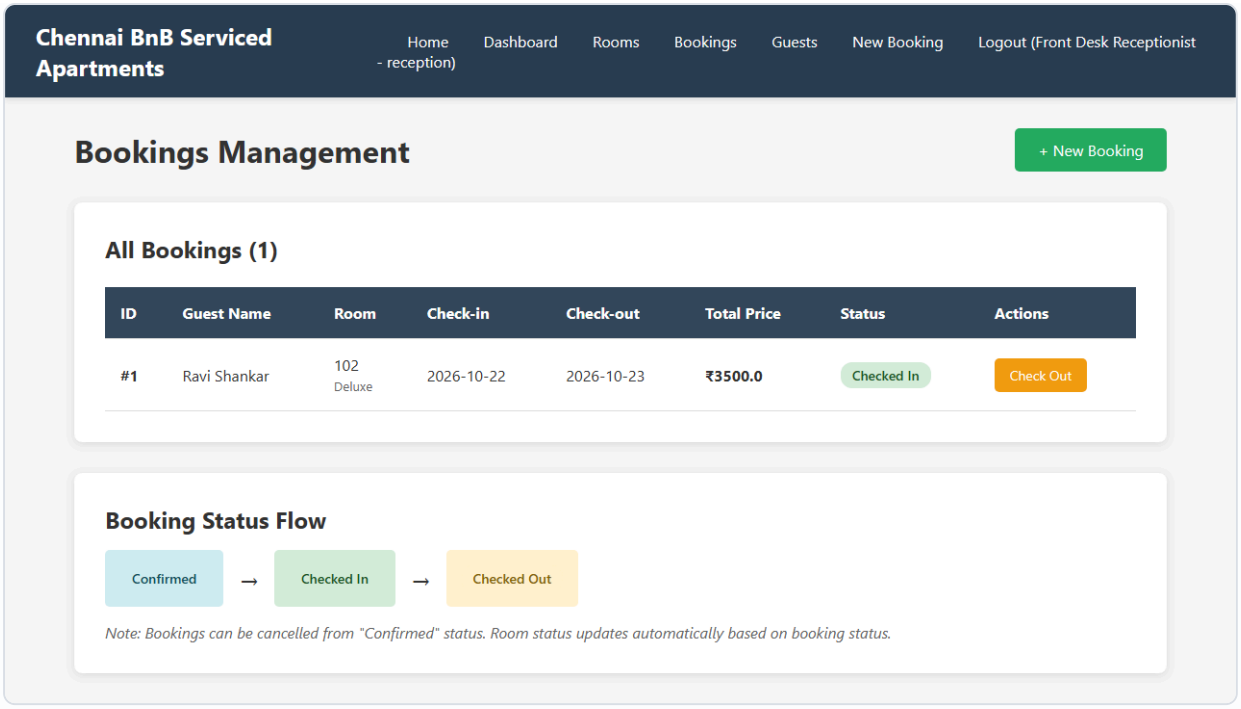
E2E-03 — Booking created

E2E-04 — Full stay lifecycle PASSED

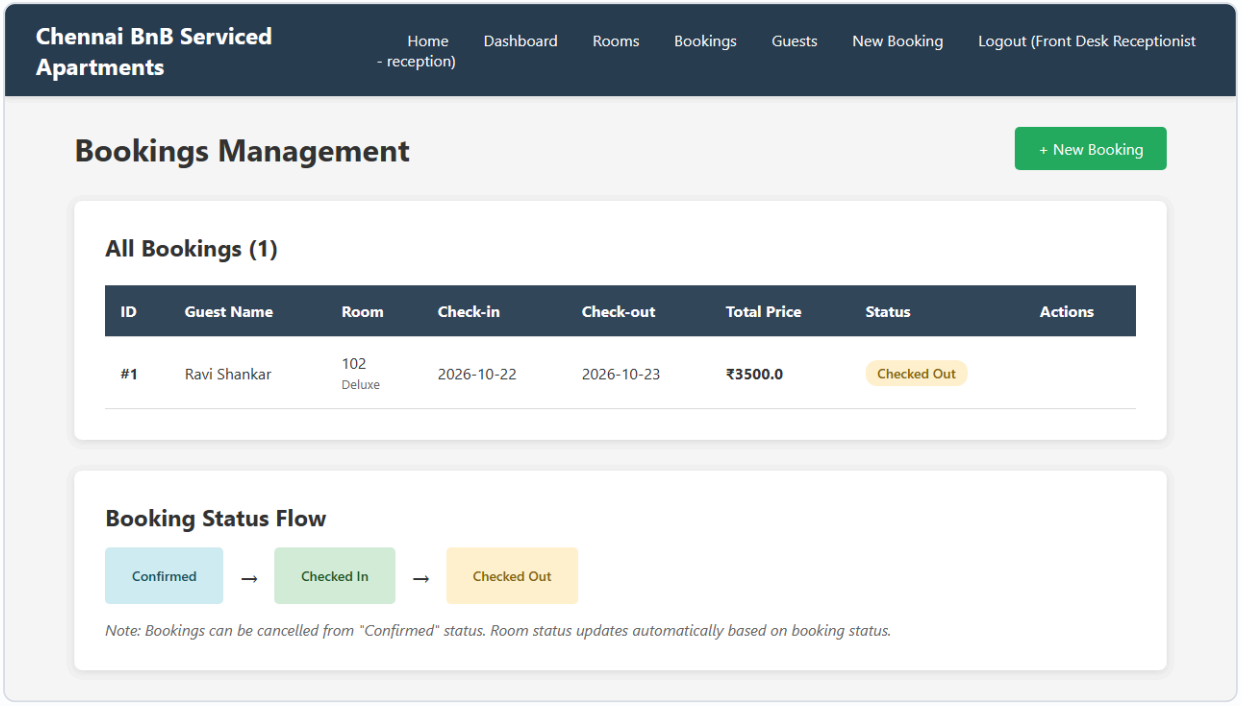
Layer: UI | confirmed -> checked_in -> checked_out, room freed at the end.



E2E-04 — Status confirmed



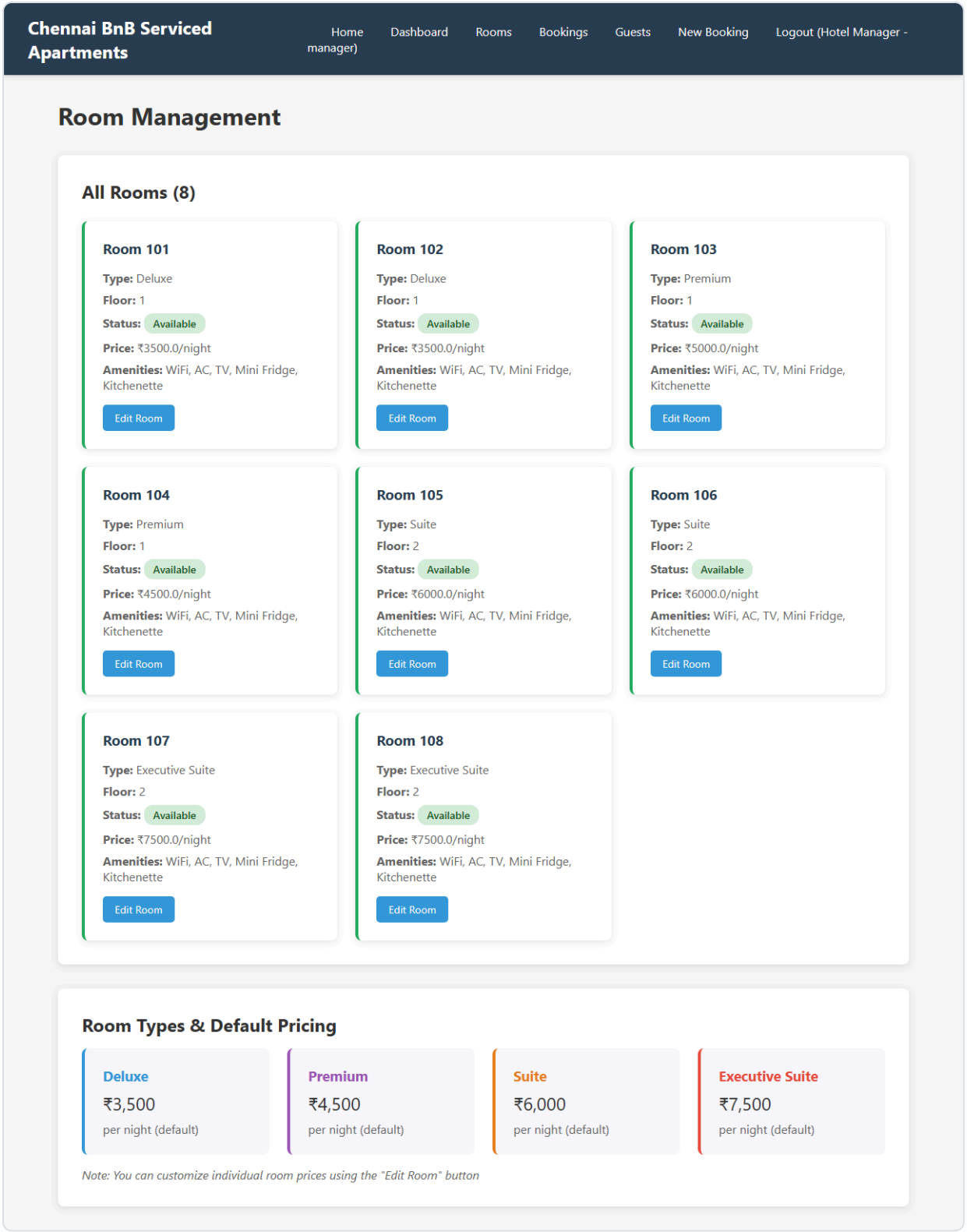
E2E-04 — Status checked in



E2E-04 — Status checked out room freed

E2E-05 — Edit room price persists PASSED

Layer: UI | A UI price change is written to and read back from MySQL.



E2E-05 — Price updated

E2E-06 — API create booking PASSED

Layer: API | POST /api/bookings returns 201 with a correct computed price.

E2E-06 API create booking — PASSED

E2E-06: A valid POST /api/bookings creates a confirmed booking.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:31",
    "guest_id": 1,
    "room_id": 1,
    "status": "confirmed",
    "total_price": 7000.0
  },
  "success": true
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 201, resp.text
assert body["success"] is True
assert booking["booking_id"] > 0
assert booking["status"] == "confirmed"
assert booking["room_id"] == 1
assert float(booking["total_price"]) == 7000.00
```

E2E-06 — Api evidence

E2E-07 — Checked-in guests report PASSED

Layer: API | A checked-in booking appears in the occupancy report.

E2E-07 Checked-in guests report — PASSED

E2E-07: A checked-in booking appears in the checked-in guests report.

HTTP EXCHANGES CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 5,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:32",
    "guest_id": 1,
    "room_id": 5,
    "status": "confirmed",
    "total_price": 12000.0
  },
  "success": true
}
```

PUT /api/bookings/1 — 200 OK

REQUEST BODY

```
{
  "status": "checked_in"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": "2026-10-21 20:30:32",
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:32",
    "guest_id": 1,
    "room_id": 5,
    "status": "checked_in",
    "total_price": 12000.0
  },
  "success": true
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

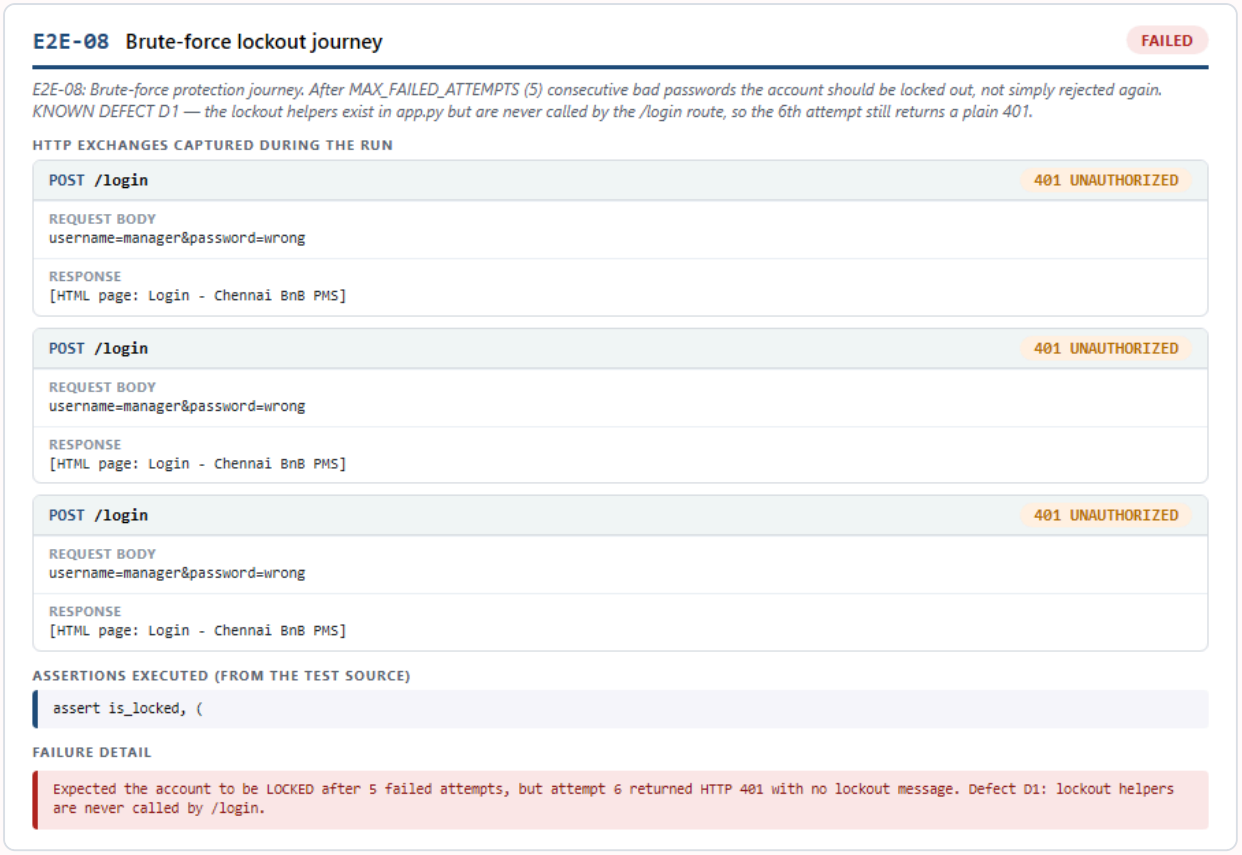
```
assert resp.status_code == 200
assert any(r["booking_id"] == bid for r in rows), "checked-in guest missing from report"
assert row["room_number"] == "105"
assert row["guest_name"] == "Test Guest"
```

E2E-07 — Api evidence

E2E-08 — Brute-force lockout journey FAILED

Layer: API | Account should lock after 5 failed logins.

> **Defect D1:** Lockout helpers `_is_login_locked()` / `_record_failed_login()` are defined in app.py but never called by the `/login` route, so no lockout ever triggers.



E2E-08 — Api evidence

Category 2 — Boundary & Edge Cases

Field validation, minimum/maximum inputs, malformed data and error handling.

ID	Test case	Layer	Result
BND-01	Login wrong password	API	PASSED
BND-02	Open-redirect guard	API	PASSED
BND-03	Check-out equals check-in	API	PASSED
BND-04	Check-out before check-in	API	PASSED
BND-05	Invalid date format	API	FAILED
BND-06	Missing required field	API	PASSED

BND-07	Non-existent room	API	PASSED
BND-08	Double-booking overlap	API	PASSED
BND-09	Price zero rejected	API	PASSED
BND-10	Room number uniqueness	DB	PASSED
BND-11	Oversized input	API	FAILED
BND-12	Blank required field	API	FAILED
BND-13	Booking form reversed dates	UI	PASSED

BND-01 — Login wrong password PASSED

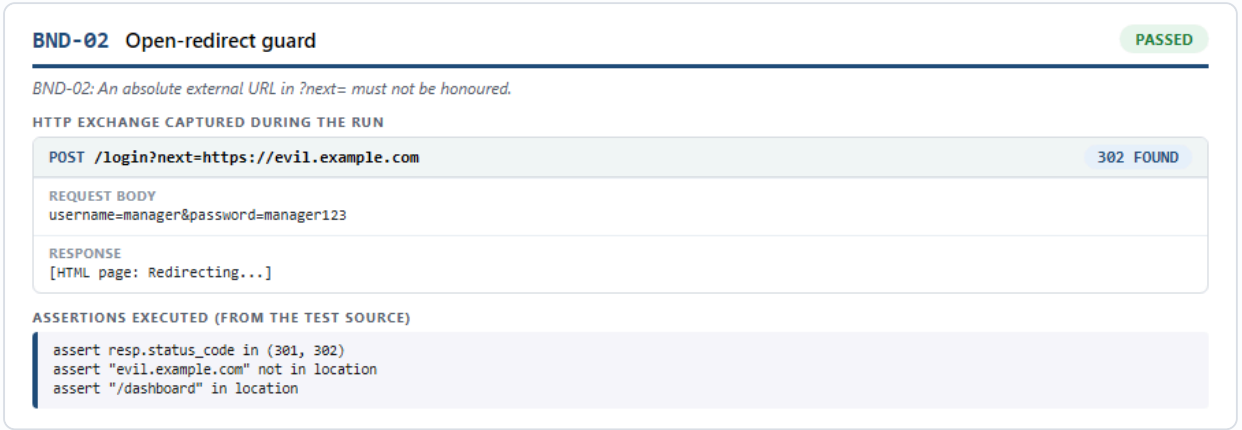
Layer: API | Bad password is rejected with HTTP 401.



BND-01 — Api evidence

BND-02 — Open-redirect guard PASSED

Layer: API | External URL in ?next= is not honoured after login.



BND-02 — Api evidence

BND-03 — Check-out equals check-in PASSED

Layer: API | Zero-night booking rejected with 400.

BND-03 Check-out equals check-in — PASSED

BND-03: Zero-night booking (check-out == check-in) is rejected.

HTTP EXCHANGE CAPTURED DURING THE RUN

`POST /api/bookings` — 400 BAD REQUEST

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-23",
  "check_out": "2026-10-23"
}
```

RESPONSE

```
{
  "error": "Check-out must be after check-in",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400
assert "check-out must be after check-in" in resp.json()["error"].lower()
```

BND-03 — Api evidence

BND-04 — Check-out before check-in PASSED

Layer: API | Negative-night booking rejected with 400.

BND-04 Check-out before check-in — PASSED

BND-04: Negative-night booking (check-out before check-in) is rejected.

HTTP EXCHANGE CAPTURED DURING THE RUN

`POST /api/bookings` — 400 BAD REQUEST

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-26",
  "check_out": "2026-10-23"
}
```

RESPONSE

```
{
  "error": "Check-out must be after check-in",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400
assert "check-out must be after check-in" in resp.json()["error"].lower()
```

BND-04 — Api evidence

BND-05 — Invalid date format FAILED

Layer: API | Malformed date should return a clean 400.

> **Defect D7:** The availability check parses the dates *before* the guarded try/except block, so a malformed date raises an uncaught `ValueError` and leaks an HTTP 500 instead of a clean 400.

BND-05 Invalid date format — FAILED

*BND-05: A malformed date must be rejected with a clean 400 'Invalid date format'. KNOWN DEFECT — the availability check parses the dates *before* the guarded try/except block, so a bad format raises an uncaught ValueError (HTTP 500).*

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings — 500 INTERNAL SERVER ERROR

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026/13/40",
  "check_out": "2026-12-01"
}
```

RESPONSE

```
[HTML page: 500 Internal Server Error]
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400, (
assert "invalid date format" in resp.json()["error"].lower()
```

FAILURE DETAIL

```
Expected 400 for a malformed date, got 500 (unhandled ValueError leaks a 500).
```

BND-05 — Api evidence

BND-06 — Missing required field PASSED

Layer: API | Omitted field rejected with 400.

BND-06 Missing required field — PASSED

BND-06: Omitting a required field is rejected with 400.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings — 400 BAD REQUEST

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "error": "Missing required fields",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400
assert "missing required fields" in resp.json()["error"].lower()
```

BND-06 — Api evidence

BND-07 — Non-existent room PASSED

Layer: API | Unknown room id returns 404.

BND-07 Non-existent room PASSED

BND-07: Booking a room id that does not exist returns 404.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings 404 NOT FOUND

REQUEST BODY

```
{
  "room_id": 9999,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "error": "Room not found",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 404
assert "room not found" in resp.json()["error"].lower()
```

BND-07 — Api evidence

BND-08 — Double-booking overlap PASSED

Layer: API | Overlapping date range on one room is rejected.

BND-08 Double-booking overlap — PASSED

BND-08: An overlapping date range on the same room is rejected.

HTTP EXCHANGES CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-25"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-25",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:47",
    "guest_id": 1,
    "room_id": 1,
    "status": "confirmed",
    "total_price": 10500.0
  },
  "success": true
}
```

POST /api/bookings — 400 BAD REQUEST

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-23",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "error": "Room not available",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert first.status_code == 201
assert overlap.status_code == 400
assert "not available" in overlap.json()["error"].lower()
```

BND-08 — Api evidence

BND-09 — Price zero rejected PASSED

Layer: API | base_price of 0 rejected with 400.

BND-09 Price zero rejected — PASSED

BND-09: A base price of 0 is rejected.

HTTP EXCHANGE CAPTURED DURING THE RUN

`PUT /api/rooms/1` — 400 BAD REQUEST

REQUEST BODY

```
{
  "base_price": 0
}
```

RESPONSE

```
{
  "error": "Price must be greater than 0",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400
assert "greater than 0" in resp.json()["error"].lower()
```

BND-09 — Api evidence

BND-10 — Room number uniqueness PASSED

Layer: DB | UNIQUE constraint on room_number is enforced.

BND-10 Room number uniqueness — PASSED

BND-10: The UNIQUE constraint on rooms.room_number is enforced by the DB.

VERIFICATION METHOD

No HTTP call - this test asserts directly against the MySQL database.

BND-10 — Api evidence

BND-11 — Oversized input FAILED

Layer: API | A 200-char name should be handled, not crash the server.

> **Defect D8:** The guest INSERT is unguarded, so MySQL strict mode raises *Data too long* and the request returns HTTP 500 rather than a validation error.

BND-11 Oversized input — FAILED

BND-11: A guest name far longer than VARCHAR(100) must be handled gracefully (rejected with 4xx), not crash the server. KNOWN DEFECT — the insert is unguarded, so MySQL strict mode raises 'Data too long' and the request returns HTTP 500.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings — 500 INTERNAL SERVER ERROR

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name":
"AAAA
```

BND-12 — Blank required field FAILED

Layer: API | A present-but-empty field should be rejected.

> **Defect D6:** The API only checks that required *keys exist*, not that they hold a value, so a blank guest name is accepted and a booking is created.

BND-12 Blank required field — FAILED

BND-12: A present-but-blank required field must be rejected. KNOWN DEFECT D6 — the API only checks that the key exists, not that it holds a value, so a blank guest name is accepted and a booking is created.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:47",
    "guest_id": 1,
    "room_id": 1,
    "status": "confirmed",
    "total_price": 7000.0
  },
  "success": true
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400, (
```

FAILURE DETAIL

```
Expected 400 for a blank guest_name, got 201 (API checks key presence only, not that the value is non-empty). Defect D6.
```

BND-12 — Api evidence

BND-13 — Booking form reversed dates PASSED

Layer: UI | UI form rejects check-out before check-in. (FIXED)

BND-13 Booking form reversed dates — PASSED

BND-13: The /new-booking form rejects check-out before check-in. (FIXED: D2)

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /new-booking — 400 BAD REQUEST

REQUEST BODY

```
guest_name=Reverse+Date&guest_email=rev%40example.com&guest_phone=9876543210&room_id=1&check_in=2026-10-26&check_out=2026-10-23
```

RESPONSE

```
[HTML page: New Booking - Chennai BnB PMS]
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 400
assert "check-out must be after check-in" in resp.text.lower()
assert query_one("SELECT COUNT(*) AS c FROM bookings")["c"] == 0
```

BND-13 — Api evidence

Category 3 — Integration Points

Data flow between frontend, API and database, plus role-based access control.

ID	Test case	Layer	Result
INT-01	Booking persists to database	API-DB	PASSED
INT-02	Booking sets room occupied	API-DB	PASSED
INT-03	Check-out frees the room	API-DB	PASSED
INT-04	Total price computed	API-DB	PASSED
INT-05	UI write visible via API	UI-DB-API	PASSED
INT-06	Auditor cannot mutate	API	PASSED
INT-07	Unauthenticated blocked	API	PASSED
INT-08	Room update persists	API-DB	FAILED
INT-09	Booking creation requires auth	API	FAILED

INT-01 — Booking persists to database PASSED

Layer: API-DB | API-created booking is readable from MySQL.

INT-01 Booking persists to database — PASSED

INT-01: A booking created via the API is readable back from the database.

HTTP EXCHANGES CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:48",
    "guest_id": 1,
    "room_id": 1,
    "status": "confirmed",
    "total_price": 7000.0
  },
  "success": true
}
```

GET /api/bookings/1 — 200 OK

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:48",
    "guest_email": "test.guest@example.com",
    "guest_id": 1,
    "guest_name": "Test Guest",
    "guest_phone": "9876543210",
    "room_id": 1,
    "room_number": "101",
    "room_type": "Deluxe",
    "status": "confirmed",
    "total_
... (truncated)
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert row is not None, "booking was not written to the database"
assert row["room_id"] == 1
assert str(row["check_in"]) == created["check_in"]
assert str(row["check_out"]) == created["check_out"]
assert fetched["booking_id"] == bid
assert fetched["room_id"] == row["room_id"]
```

INT-01 — Api evidence

INT-02 — Booking sets room occupied PASSED

Layer: API-DB | Creating a booking cascades room status to occupied.

INT-02 Booking sets room occupied — PASSED

INT-02: Creating a booking cascades the room status to 'occupied'.

HTTP EXCHANGES CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY


```
{
  "room_id": 4,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```


RESPONSE


```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:48",
    "guest_id": 1,
    "room_id": 4,
    "status": "confirmed",
    "total_price": 9000.0
  },
  "success": true
}
```


GET /api/rooms/4 — 200 OK

RESPONSE


```
{
  "data": {
    "amenities": ["WiFi", "AC", "TV", "Mini Fridge", "Kitchenette"],
    "base_price": 4500.0,
    "current_booking_id": 1,
    "current_guest": {
      "booking_id": 1,
      "check_in": "2026-10-22",
      "check_out": "2026-10-24",
      "email": "test.guest@example.com",
      "guest_id": 1,
      "name": "Test Guest",
      "phone": "9876543210"
    },
    "current_guest_id": 1,
    "floor": 1,
    "room_id"
... (truncated)
```


ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert room["status"] == "occupied"
assert room["current_booking_id"] == created["booking_id"]
assert room["current_guest_id"] == created["guest_id"]
assert api_room["status"] == "occupied"
```

INT-02 — Api evidence

INT-03 — Check-out frees the room PASSED

Layer: API-DB | Check-out cascades the room back to available.

INT-03 Check-out frees the room — PASSED

INT-03: Checking out cascades the room back to 'available'.

HTTP EXCHANGES CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 4,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:48",
    "guest_id": 1,
    "room_id": 4,
    "status": "confirmed",
    "total_price": 9000.0
  },
  "success": true
}
```

PUT /api/bookings/1 — 200 OK

REQUEST BODY

```
{
  "status": "checked_out"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": "2026-10-21 20:30:48",
    "created_at": "2026-10-21 20:30:48",
    "guest_id": 1,
    "room_id": 4,
    "status": "checked_out",
    "total_price": 9000.0
  },
  "success": true
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert query_one("SELECT status FROM rooms WHERE room_id=4")["status"] == "occupied"
assert resp.status_code == 200
assert room["status"] == "available"
assert room["current_guest_id"] is None
```

INT-03 — Api evidence

INT-04 — Total price computed PASSED

Layer: API-DB | total_price equals base price x nights.

INT-04 Total price computed — PASSED

INT-04: total_price == room base price x number of nights.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings — 201 CREATED

REQUEST BODY

```
{
  "room_id": 5,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-25"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-25",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:48",
    "guest_id": 1,
    "room_id": 5,
    "status": "confirmed",
    "total_price": 18000.0
  },
  "success": true
}
```

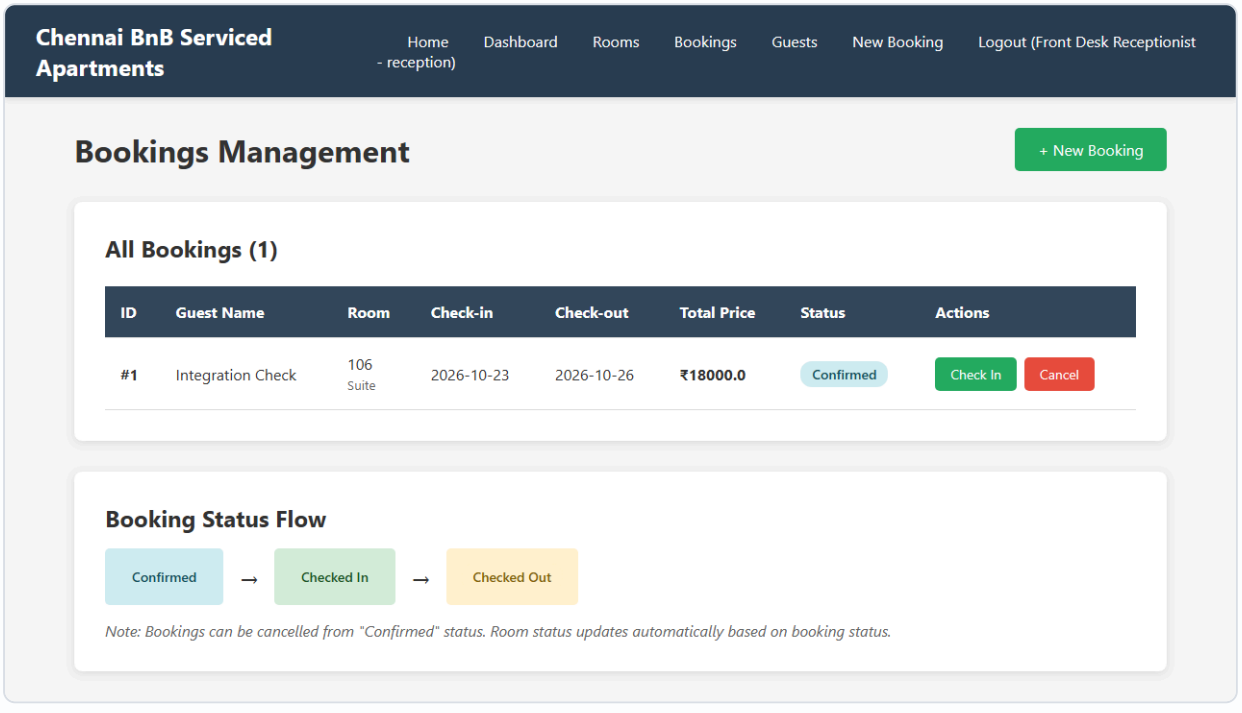
ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert float(created["total_price"]) == 18000.00
assert float(row["total_price"]) == 18000.00
```

INT-04 — Api evidence

INT-05 — UI write visible via API PASSED

Layer: UI-DB-API | A booking made in the UI is returned by the REST API.



INT-05 — Created in ui

Pretty-print

{"data":[{"booking_id":1,"check_in":"2026-10-23","check_out":"2026-10-26","checked_in_at":null,"checked_out_at":null,"created_at":"2026-10-21
20:30:45","guest_email":"integration@example.com","guest_id":1,"guest_name":"Integration
Check","guest_phone":"9845022222","room_id":6,"room_number":"106","room_type":"Suite","status":"confirmed","total_price":18000.0}],"success":true}

INT-05 — Same record via api

INT-06 — Auditor cannot mutate PASSED

Layer: API | Auditor role gets 403 on booking deletion.

INT-06 Auditor cannot mutate — PASSED

INT-06: The auditor role is forbidden from cancelling a booking.

HTTP EXCHANGES CAPTURED DURING THE RUN

`POST /api/bookings` — 201 CREATED

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:48",
    "guest_id": 1,
    "room_id": 1,
    "status": "confirmed",
    "total_price": 7000.0
  },
  "success": true
}
```

`DELETE /api/bookings/1` — 403 FORBIDDEN

RESPONSE

```
{
  "error": "Forbidden: insufficient role",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 403
assert "forbidden" in resp.json()["error"].lower()
```

INT-06 — Api evidence

INT-07 — Unauthenticated blocked PASSED

Layer: API | Guarded endpoint returns 401 without a session.

INT-07 Unauthenticated blocked — PASSED

INT-07: A guarded endpoint rejects unauthenticated access with 401.

HTTP EXCHANGE CAPTURED DURING THE RUN

`GET /api/guests` — 401 UNAUTHORIZED

RESPONSE

```
{
  "error": "Authentication required",
  "success": false
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 401
assert "authentication required" in resp.json()["error"].lower()
```

INT-07 — Api evidence

INT-08 — Room update persists FAILED

Layer: API-DB | PUT /api/rooms should persist the new price.

> **Defect D3:** `PUT /api/rooms/<id>` mutates a throwaway dictionary and never writes to MySQL, yet still reports *Room updated successfully*.

INT-08 Room update persists — FAILED

INT-08: PUT /api/rooms/<id> must persist the new price to the database. KNOWN DEFECT D3 — the handler mutates a throwaway dict and never writes to MySQL, yet still reports 'Room updated successfully'.

HTTP EXCHANGES CAPTURED DURING THE RUN

PUT /api/rooms/1 — 200 OK

REQUEST BODY

```
{
  "base_price": 9999
}
```

RESPONSE

```
{
  "data": {
    "amenities": ["WiFi", "AC", "TV", "Mini Fridge", "Kitchenette"],
    "base_price": 3500.0,
    "current_booking_id": null,
    "current_guest": null,
    "current_guest_id": null,
    "floor": 1,
    "room_id": 1,
    "room_number": "101",
    "room_type": "Deluxe",
    "status": "available"
  },
  "message": "Room updated successfully",
  "success": true
}
```

GET /api/rooms/1 — 200 OK

RESPONSE

```
{
  "data": {
    "amenities": ["WiFi", "AC", "TV", "Mini Fridge", "Kitchenette"],
    "base_price": 3500.0,
    "current_booking_id": null,
    "current_guest": null,
    "current_guest_id": null,
    "floor": 1,
    "room_id": 1,
    "room_number": "101",
    "room_type": "Deluxe",
    "status": "available"
  },
  "success": true
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 200
assert resp.json()["success"] is True
assert float(after["base_price"]) == 9999.00, (
```

FAILURE DETAIL

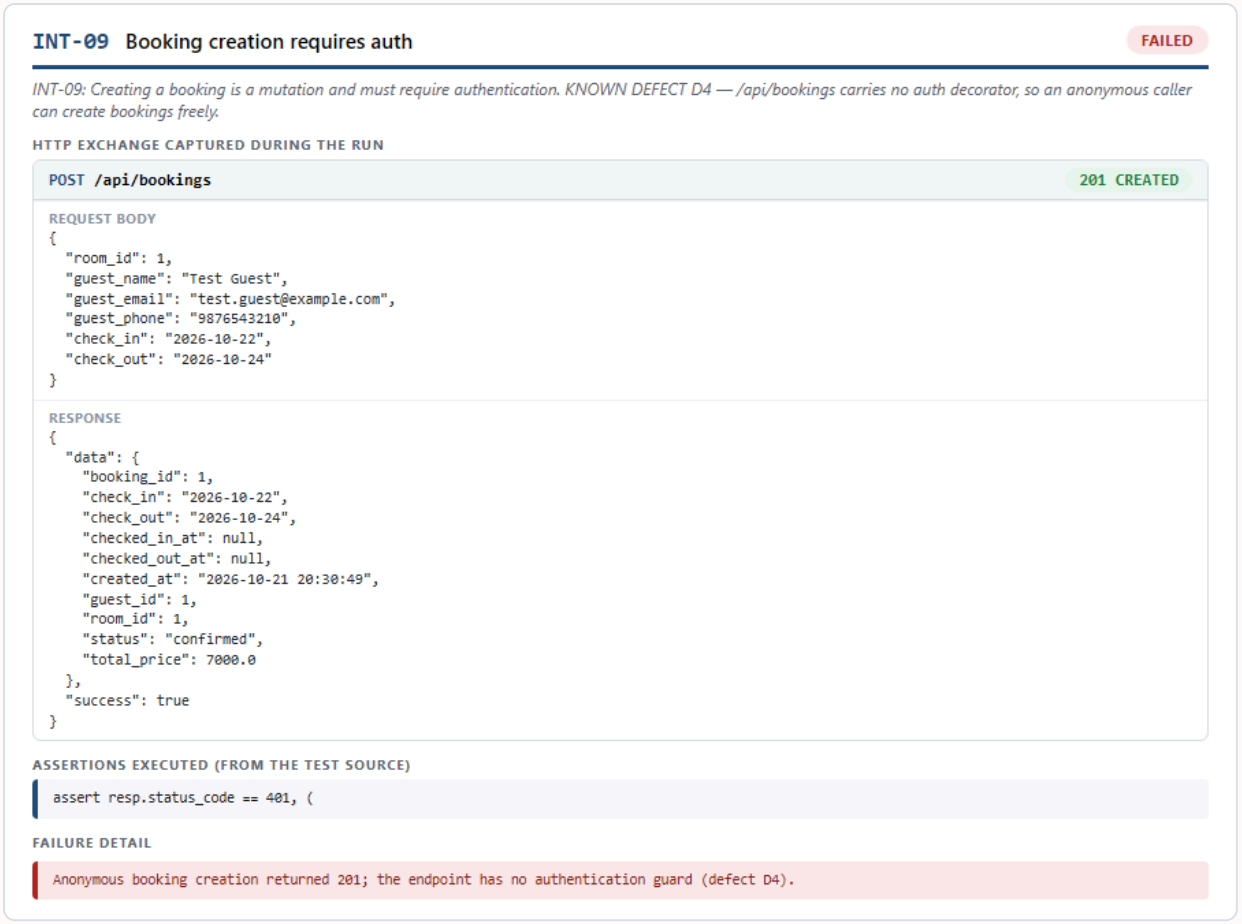
```
PUT reported success but GET still returns 3500.0; the update was never persisted to MySQL. Defect D3.
```

INT-08 — Api evidence

INT-09 — Booking creation requires auth FAILED

Layer: API | Anonymous booking creation should be blocked.

> **Defect D4:** `/api/bookings` carries no authentication decorator, so an anonymous caller can create bookings freely.



INT-09 Booking creation requires auth FAILED

INT-09: Creating a booking is a mutation and must require authentication. KNOWN DEFECT D4 — /api/bookings carries no auth decorator, so an anonymous caller can create bookings freely.

HTTP EXCHANGE CAPTURED DURING THE RUN

POST /api/bookings 201 CREATED

REQUEST BODY

```
{
  "room_id": 1,
  "guest_name": "Test Guest",
  "guest_email": "test.guest@example.com",
  "guest_phone": "9876543210",
  "check_in": "2026-10-22",
  "check_out": "2026-10-24"
}
```

RESPONSE

```
{
  "data": {
    "booking_id": 1,
    "check_in": "2026-10-22",
    "check_out": "2026-10-24",
    "checked_in_at": null,
    "checked_out_at": null,
    "created_at": "2026-10-21 20:30:49",
    "guest_id": 1,
    "room_id": 1,
    "status": "confirmed",
    "total_price": 7000.0
  },
  "success": true
}
```

ASSERTIONS EXECUTED (FROM THE TEST SOURCE)

```
assert resp.status_code == 401, (
```

FAILURE DETAIL

```
Anonymous booking creation returned 201; the endpoint has no authentication guard (defect D4).
```

INT-09 — Api evidence

6. Defects Found (open)

These defects remain in the build and are what the failing tests detect.

Ref	Test	Defect
D1	`E2E-08`	Lockout helpers `_is_login_locked()` / `_record_failed_login()` are defined in app.py but never called by the `/login` route, so no lockout ever triggers.
D3	`INT-08`	`PUT /api/rooms/<id>` mutates a throwaway dictionary and never writes to MySQL, yet still reports *Room updated successfully*.

D4	`INT-09`	`/api/bookings` carries no authentication decorator, so an anonymous caller can create bookings freely.
D6	`BND-12`	The API only checks that required *keys exist*, not that they hold a value, so a blank guest name is accepted and a booking is created.
D7	`BND-05`	The availability check parses the dates *before* the guarded try/except block, so a malformed date raises an uncaught `ValueError` and leaks an HTTP 500 instead of a clean 400.
D8	`BND-11`	The guest INSERT is unguarded, so MySQL strict mode raises *Data too long* and the request returns HTTP 500 rather than a validation error.

7. Defects Found & Fixed

These were discovered while building the suite and have been repaired. Their tests now pass, which is the evidence that the fix works.

D2 — `/new-booking` form validation
The booking form had no validation at all: a malformed date crashed with a 500 and reversed dates created a booking with a negative total. It now validates required fields, date format, date ordering and room availability.
Verified by: BND-13, E2E-03, E2E-04, INT-05

D5 — `/rooms/edit` persistence & validation
The room edit form mutated a throwaway dictionary and saved nothing, and accepted any price. It now writes through to MySQL and rejects a price of zero or less.
Verified by: E2E-05

D9 — Database connection leak on error
`create_guest()` and `create_booking()` never closed their connection when an INSERT failed, leaving an open transaction holding table locks. Both now release the connection in a `finally` block.
Verified by: Prevented the suite from hanging on BND-11

8. How to Run the Demo

#	Step	Command
1	Ensure MySQL is running and the schema is loaded	`mysql -u root -p < schema.sql`
2	Run the whole suite (starts the app automatically)	`python -m pytest -v`

3	Run one category only	`python -m pytest tests/test_category2_boundary.py -v`
4	Run only the browser tests (add `--headed` to watch)	`python -m pytest tests/test_category1_e2e_ui.py --headed`
5	Regenerate the HTML report	`python -m pytest --html=test_evidence/report.html --self-contained-html`

6	Rebuild this document (after a run)	`python tools/capture_evidence.py`
		`python tools/render_api_cards.py`
		`python tools/build_demo_document.py`

> **Live demo tip:** run step 4 with `--headed --slowmo 600` to show Chromium driving the application in real time - filling the booking form, checking a guest in and checking them out - then run step 2 to show the full suite and the defects it catches.